

TECHNICAL ARCHITECTURE & INTERVIEW PREPARATION GUIDE

Executive Portfolio Architecture & Technical Interview Q&A; for Dubai Recruiter Standards

Candidate: Kannan Appiya Santharam | **Role Target:** Senior Lead Software Engineer / Engineering Manager
Relocation Status: Ready to Relocate to Dubai, UAE | **Notice Period:** 60 Days | **Tech Stack:** React 19, TypeScript, HTTP Streamable, Rspack

1. SYSTEM ARCHITECTURE & TECHNICAL SPECIFICATIONS

This executive portfolio was built to present Kannan Appiya Santharam's 10.5+ years of monorepo leadership, build acceleration metrics, and AI-native engineering capabilities directly to top UAE technology recruiters and hiring managers.

Component Layer	Technology & Implementation Specification
Framework & Core	React 19 (Latest Stable) + React Compiler (babel-plugin-react-compiler via @rolldown/plugin-babel). Auto-memoizes JSX render trees without manual useMemo/useCallback.
Build Engine	Vite 8 + TypeScript 6. Production client bundle built in 390ms.
Streaming Architecture	HTTP Streamable Web Streams (fetch + ReadableStream). Streams real-time LLM tokens and AI agent tool calls with zero text-framing overhead (~80ms TTFT).
Styling System	Tailwind CSS v4 + Custom CSS Variable Engine (--bg-page, --bg-card, --text-title, --text-body). Supports seamless 100% Light and Dark theme switching.
Package Manager	pnpm (v11.20.0) with pnpm-lock.yaml. Content-addressable store for fast, deterministic installs (4.8s).
Mobile Responsiveness	Mobile-first design with swipeable experience tab strips, touch-friendly 44px+ tap targets, and responsive hero photo ordering.

2. INTERVIEW PREPARATION: RESUME & PORTFOLIO IMPACT WINS (PART A)

Technical recruiters, Engineering Managers, and Software Architects in Dubai will dive deep into the specific achievements listed on Kannan's portfolio. Below are the exact technical questions and model answers.

Q1: You executed a solo Webpack 5 to Rspack migration in 3 weeks cutting build time by 96% (2 minutes to 5 seconds across 12 packages). How did you execute this safely?

Context: The SuperOps monorepo contained 12 packages with heavy compile bottlenecks caused by ts-loader, babel-loader, and complex Webpack plugin chains.

Execution Steps:

- Analyzed monorepo dependency graph and identified Webpack loader bottlenecks.
- Replaced Webpack 5 with Rust-powered **Rspack** and built-in **swc-loader** for ultra-fast compilation.
- Configured Rspack Module Federation for shared core UI components.
- **Quantitative Result:** Cold-start compilation dropped from 120 seconds to 5 seconds (96% speedup). Hot Module Replacement (HMR) re-compiles in under 50ms.
- **Impact:** Saved hundreds of engineering hours monthly across distributed teams and accelerated CI/CD build pipelines in Jenkins.

Q2: How did you architect the AI test-authoring platform over HTTP Streamable Web Streams?

Context: Traditional manual end-to-end Playwright/Cypress test writing was slow and brittle during UI feature releases.

Architecture: Built a React 19 frontend paired with a Node.js orchestration layer streaming LLM tool calls in real time over **HTTP Streamable Web Streams (fetch + ReadableStream)**.

Self-Healing Loop: LLM agent inspects DOM tree snapshots, auto-generates Playwright spec code, executes the test, captures assertion failures, and self-heals broken selectors automatically.

Q3: How did you author custom production Claude Code skills using MCP servers and enforce write-scope guardrails?

Model Context Protocol (MCP): Implemented custom TypeScript MCP servers that expose codebase AST symbol graphs, imports, and component hierarchies directly to Claude Code and Cursor IDE.

Write-Scope Guardrails: Engineered multi-layered safety guardrails. Autonomous AI coding agents are permitted to modify target feature components (e.g., `src/features/**`) while strictly blocking edits to core system contracts, GraphQL schemas, or global design tokens (e.g., `src/core/**`).

Q4: How do you protect platform quality with a 232-spec Playwright regression suite and Knip static analysis?

Knip Static Analysis: Executed Knip across the monorepo, identifying and stripping over **30,000 lines of dead code**, unused exports, and unreferenced dependencies.

Playwright Suite: Maintained 232 Playwright end-to-end specs structured using Page Object Models (POM), parallel execution workers, and visual regression snapshot testing in Jenkins CI.

3. INTERVIEW PREPARATION: PORTFOLIO ENGINEERING & SOFTWARE STANDARDS (PART B)

Interviewers will also examine the engineering techniques used to build this portfolio itself to gauge React 19 knowledge, CSS architecture, performance, and package management.

Q5: Why did you build this application with React 19 and React Compiler instead of traditional React 18 with useMemo/useCallback?

Answer: React 19's babel-plugin-react-compiler automatically memoizes component render trees, JSX elements, and hook dependencies at build time. It eliminates manual useMemo and useCallback boilerplate while preventing unnecessary component re-renders. This demonstrates forward-thinking adoption of React 19's production compiler pipeline.

Q6: Why is HTTP Streamable (fetch + ReadableStream) superior to standard SSE or REST JSON for LLM chatbots?

Answer: HTTP Streamable eliminates line-by-line SSE data framing overhead (no data: {...} string regex parsing needed). Chunks stream directly over native browser WHATWG ReadableStream readers, cutting Time-To-First-Token (TTFT) by 90% down to ~80ms and streaming tokens in real time.

Q7: How did you implement 100% Light & Dark theme support without layout shifts or text contrast issues?

Answer: Constructed a CSS custom property token engine in index.css mapped under html.dark and html.light selectors. Theme state is persisted in localStorage via React ThemeContext. Replaced hardcoded utilities with semantic CSS variables (var(--bg-page), var(--bg-card), var(--text-title), var(--text-body)), ensuring 100% theme coverage across all 7 sections with zero text contrast degradation.

Q8: Why did you switch package management from npm to pnpm?

Answer: pnpm uses a content-addressable global store with hard links, saving disk space and speeding up installs to 4.8 seconds. It enforces strict non-hoisted dependency resolution, preventing hidden 'ghost dependency' bugs common in npm/yarn.

Q9: How did you optimize the recruiter experience for mobile viewports?

Answer: Mobile-first photo placement brings the candidate portrait and relocation status (Dubai, UAE) to the top of mobile screens. Added a horizontally swipeable company tab strip for career history, 44px+ touch-friendly buttons, and direct one-click PDF resume download triggers.

Executive Summary for Dubai Hiring Managers: Kannan Appiya Santharam brings 10.5+ years of proven software architecture, monorepo performance gains (96% compilation speedup), and production AI-native engineering (HTTP Streamable Web Streams, Claude Code MCP servers). Ready to relocate immediately to Dubai, UAE on 60 days notice.